Advanced Algorithms for Machine Learning

Assignment 1: Shuttle MDP, REINFORCE, and Variance Reduction

GitHub username: **Eddiesg666**

Repository: **IEMS469-Assignment1**

Source code: https://github.com/Eddiesg666/IEMS469-Assignment1

The report uses expected discounted COST for the shuttle model and total episode REWARD for reinforcement-learning games. In the shuttle model lower is better; in the games higher is better.

1. Execution instructions

Set up Python 3.9 or newer. Create and activate a virtual environment; install dependencies; then run the commands below from the repository root.

```
python3 -m venv .venv

source .venv/bin/activate

python -m pip install -r requirements.txt

python -m pytest -q tests/

python shuttle.py --out . --mc-episodes 1000

python pg.py train --game cartpole --baseline none --episodes 1500 --eval-episodes 500

python pg.py train --game cartpole --baseline running --episodes 1500 --eval-episodes 500
```

For each reward-only Pong run, use the same shared configuration: `--episodes 200 --batch-episodes 2 --pong-network geometry --pong-observation stackmotion --pong-guide-coef 0 --pong-update-mode episode --pong-return-mode episode --entropy-coef 0 --lr 0.0003 --eval-stochastic --eval-episodes 500`. Run this once with `--baseline none` and once with `--baseline running`. The real experimental run directories are runs/pong_none and runs/pong_running.

```
python compare.py
```

Each of the four required policies has been evaluated on 500 games, with a per-episode CSV, plots, configuration, and summary JSON. The optional guided-Pong extension is described separately in Section 5. Training seeds were kept equal across the two methods within each environment; one training seed per method limits claims about generalization.

2. Shuttle dispatching MDP (30 points)

Model and assumptions

State s in {0,...,200} is the current passenger queue; action a in {0,1} means wait or dispatch one shuttle. Dispatching removes at most K=15 passengers. New arrivals A are independent and uniform on {1,2,3,4,5}. We decide before arrivals, assess holding cost after dispatch and before arrivals, and cap the next queue at 200. Passengers arriving beyond the cap are excluded from the modeled queue. A dispatched shuttle costs 100 even when partially filled. These assumptions resolve timing and overflow details not explicitly fixed by the prompt.

```
remaining(s,a) = max(0, s - 15*a)

s_next = min(200, remaining(s,a) + A)
```

```
cost(s,a) = 100*a + 2*remaining(s,a)

V*(s) = min_a { cost(s,a) + 0.95*(1/5)*sum[j=1..5] V*(s_next) }
```

Finite-horizon enumeration (T=500): set V_500(s)=0 and enumerate each possible action and arrival outcome backwards from t=499 to 0. Infinite-horizon value iteration applies the Bellman optimality operator until the contraction bound gives a value error below the tolerance. Policy iteration evaluates the current policy with the linear system (I - gamma P_pi)V_pi = c_pi and greedily improves it until stable.

Results

Metric	Verified numeric result
Minimum dispatch queue	13
Dispatch states	13 to 200
Value iteration count	650
Policy iteration count	6
V*(0)	576.6702
V*(15)	676.6702
V*(40)	893.2614
V*(200)	3821.5865

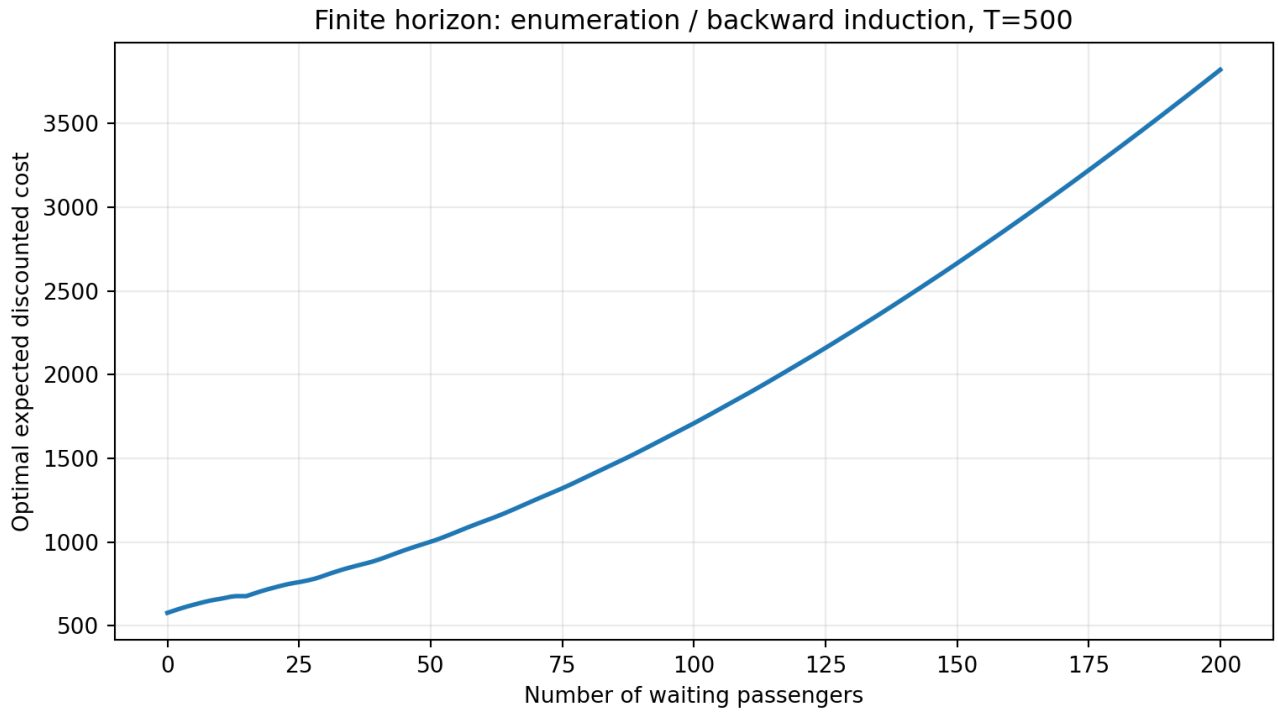


Figure 1. Optimal finite-horizon cost-to-go V_0(s), T=500.

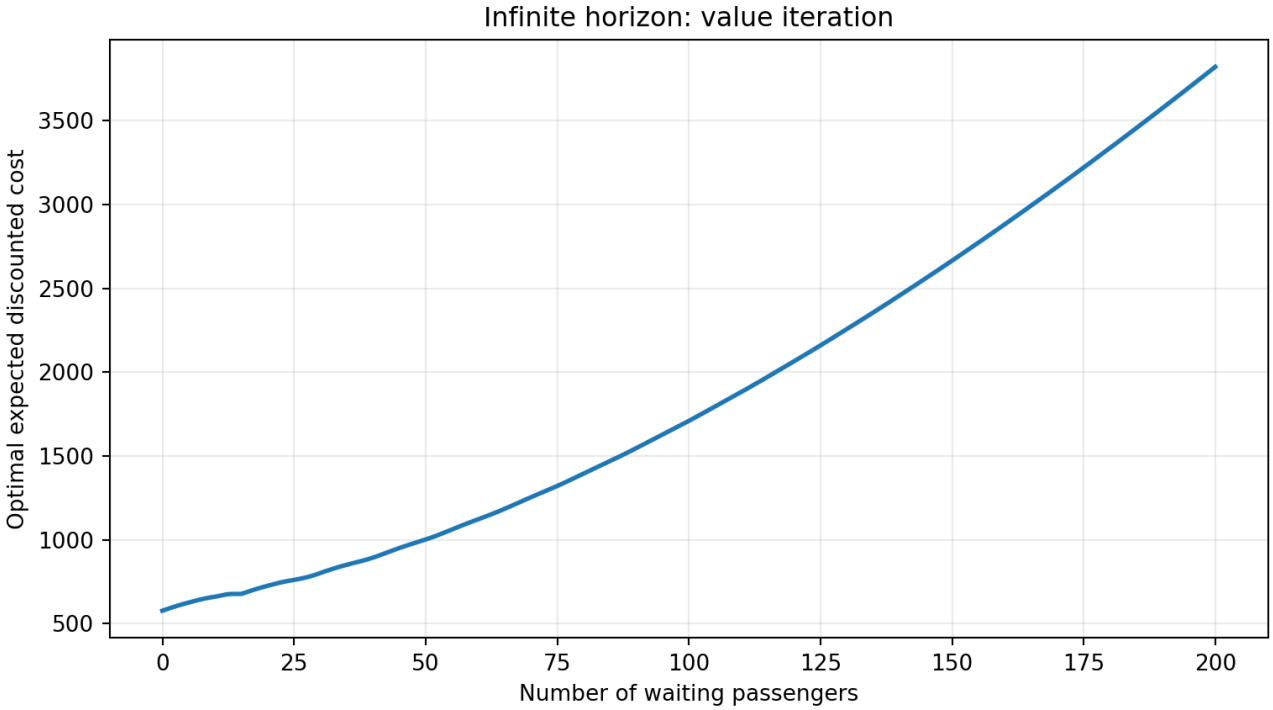


Figure 2. Infinite-horizon optimal cost V*(s) from value iteration.

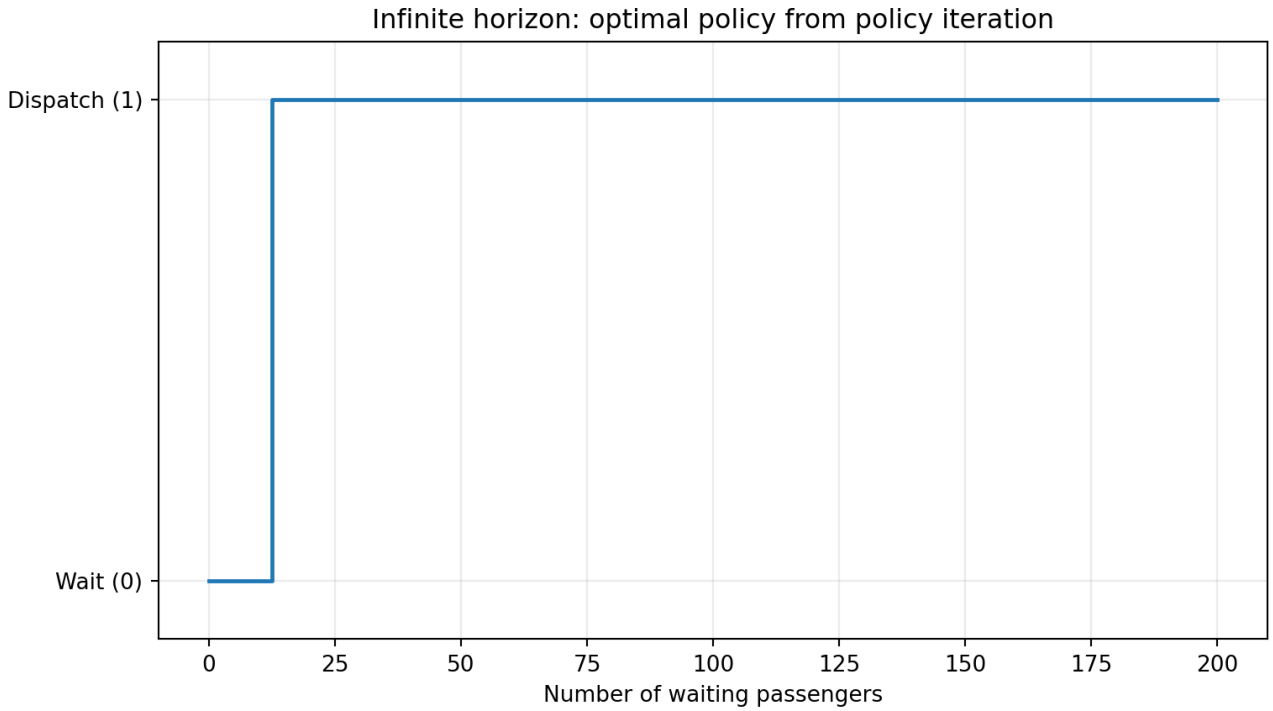


Figure 3. Policy iteration: wait at 0-12 and dispatch at 13-200.

Controlled model-sensitivity experiment

Perturbation	Dispatch threshold	V*(40)
default	13	893.26
less expensive shuttle	11	570.69
more expensive shuttle	14	1504.21
higher waiting cost	10	1252.60
lower waiting cost	14	752.10
free shuttle	1	77.23

A cheaper shuttle reduces the threshold from 13 to 11; a more expensive shuttle raises it to 14. Raising the per-passenger holding cost from 2 to 5 lowers the threshold to 10; halving it raises the threshold to 14. These controlled perturbations match the economic predictions of the MDP and provide validation beyond checking matrix sizes or code execution.

Edge cases and experimental validation

Transition probabilities were verified to sum to one at every state-action pair. Overflow at queue 200 collapses all excess-arrival outcomes onto state 200. A zero queue is never dispatched under the specified costs. With arrivals fixed to zero, state 0 is absorbing and has zero value. For gamma=0, the dynamic-programming values equal minimum immediate costs. With dispatch cost zero, every positive queue selects dispatch. The optimal value is nondecreasing in queue length.

Numerical validation	Measured result
Bellman residual max norm	9.095e-13
|V_finite - V_VI| max	4.938e-09
|V_PI - V_VI| max	2.728e-12
Policy agreement across all states	True

Policy from initial state s=40	Simulated mean discounted cost
Optimal	894.36 (95% CI 892.39 to 896.34)
Always wait	3734.81
Always dispatch	2077.44

Each shuttle policy was simulated over 1000 independent arrival paths with a 500-step horizon. The optimal-policy sample mean 894.36 agrees with the Bellman value 893.26 within sampling uncertainty. This tests the actual stochastic simulator against a separately computed Bellman solution, not merely that code runs without crashing.

3. Policy gradient: Part 1 (40 points)

Objective and implementation

For each episode we collect states s_t, sampled actions a_t, and immediate rewards r_t. The policy network produces logits for the two permitted actions. Actions are sampled from the resulting categorical distribution during training. The REINFORCE reward-to-go estimate and loss are:

```
G_t = r_t + gamma*r_(t+1) + ... + gamma^(T-t-1)*r_(T-1)

L(theta) = -(1/N) * sum_{all collected t} log pi_theta(a_t|s_t) * G_t
```

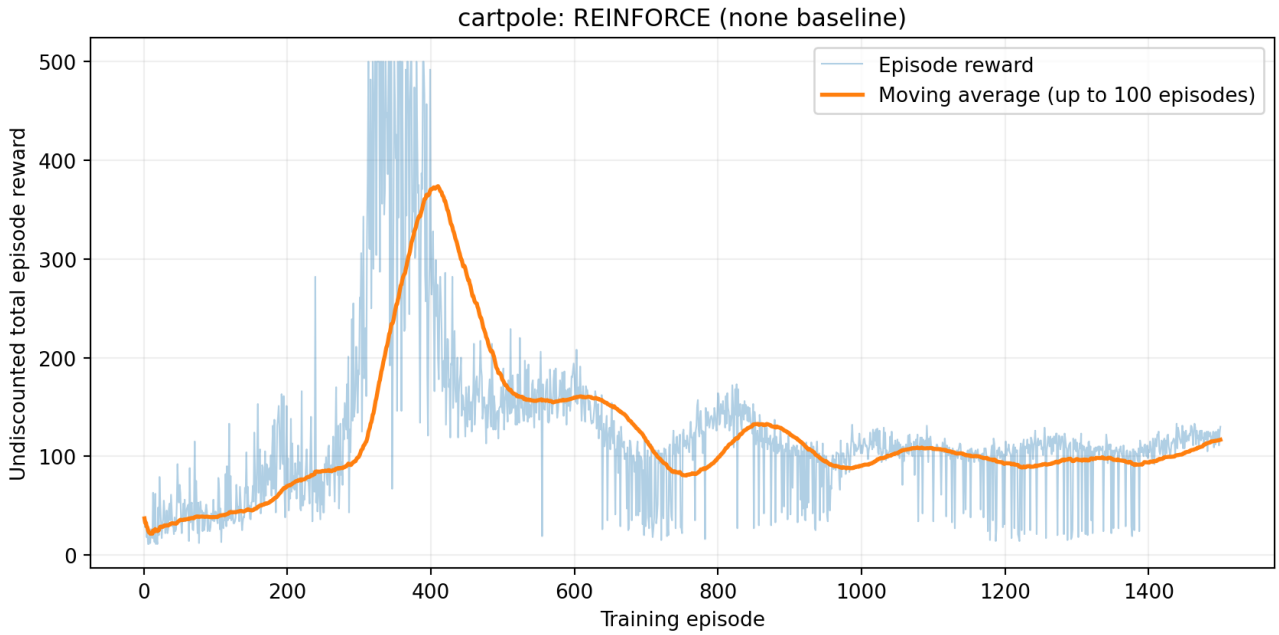
Training uses Adam, gradients averaged across collected transitions, gradient norm clipping at 10, and full episodes; samples are never mixed across episode boundaries when computing discounted returns. Results and reward curves report UNDISCOUNTED episode scores, matching gameplay interpretation, while optimization uses discounted reward-to-go. This is the standard reward-to-go REINFORCE estimator; it omits an additional outer gamma^t factor that would correspond to the exact gradient of a discounted-from-start objective.

CartPole-v1 uses a 4-to-128-to-2 ReLU MLP, discount 0.95, and Adam learning rate 0.01. For the final Pong comparison, both policies use the same geometry-aware visual network: successive preprocessed 80x80 binary frames plus signed frame motion, a small convolutional image branch, and a hand-engineered geometry branch extracting ball and paddle coordinates from those image frames. The result feeds a two-action MLP. This architecture is shared across the two reward-only conditions, and does not access game RAM. Both reward-only Pong runs use discount 0.99, Adam rate 0.0003, 200 complete training episodes, 2 episodes per update, full-game reward-to-go, and NO auxiliary visual guidance or entropy bonus. The modest 200-episode budget is a computational constraint, not evidence of convergence.

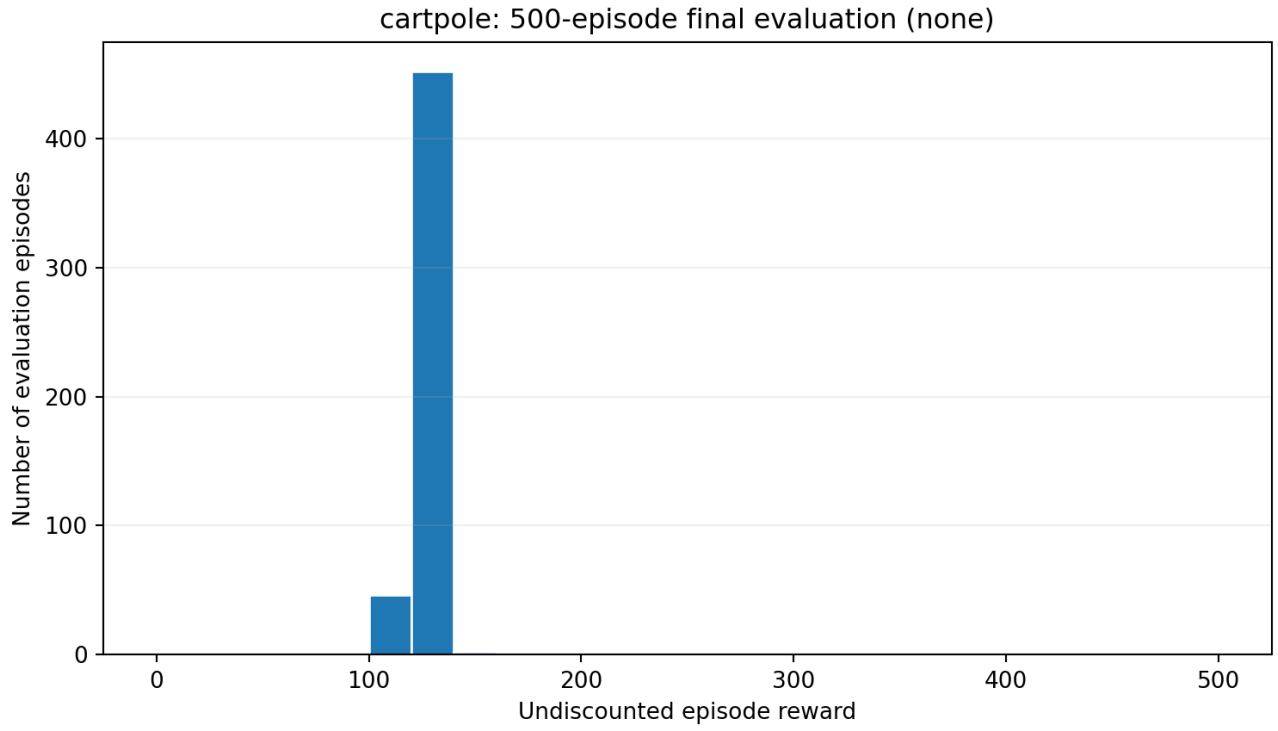
For Pong, network index 0 maps to Atari action 2 (RIGHT), and network index 1 maps to action 3 (LEFT). A fixed FIRE (action 1) is sent only at the beginning of a new game to initiate gameplay; this is not an action learned or selected by the network. The --no-start-fire option disables this initialization if prohibited.

Cartpole: without baseline

Experimental setting	Observed result
Environment	CartPole-v1
Training episodes	1500
Discount gamma	0.95
Adam learning rate	0.01
First 100 training episodes, mean reward	38.410
Last 100 training episodes, mean reward	116.740
500-episode evaluation mean	126.624
500-episode evaluation sample std	5.694
Evaluation action method	deterministic argmax of two-action policy
95% normal CI for evaluation mean	126.125 to 127.123
Artificially capped training / evaluation games	0 / 0



cartpole: reward per training episode and 100-episode moving average.

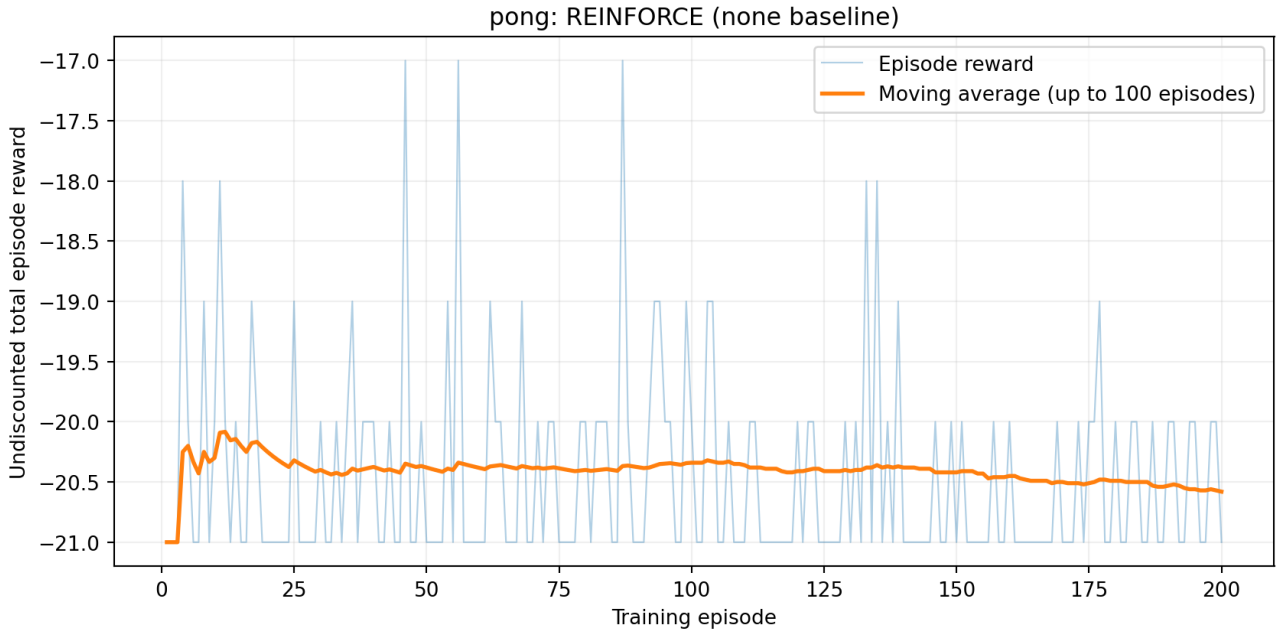


cartpole: empirical score distribution of 500 final policy rollouts.

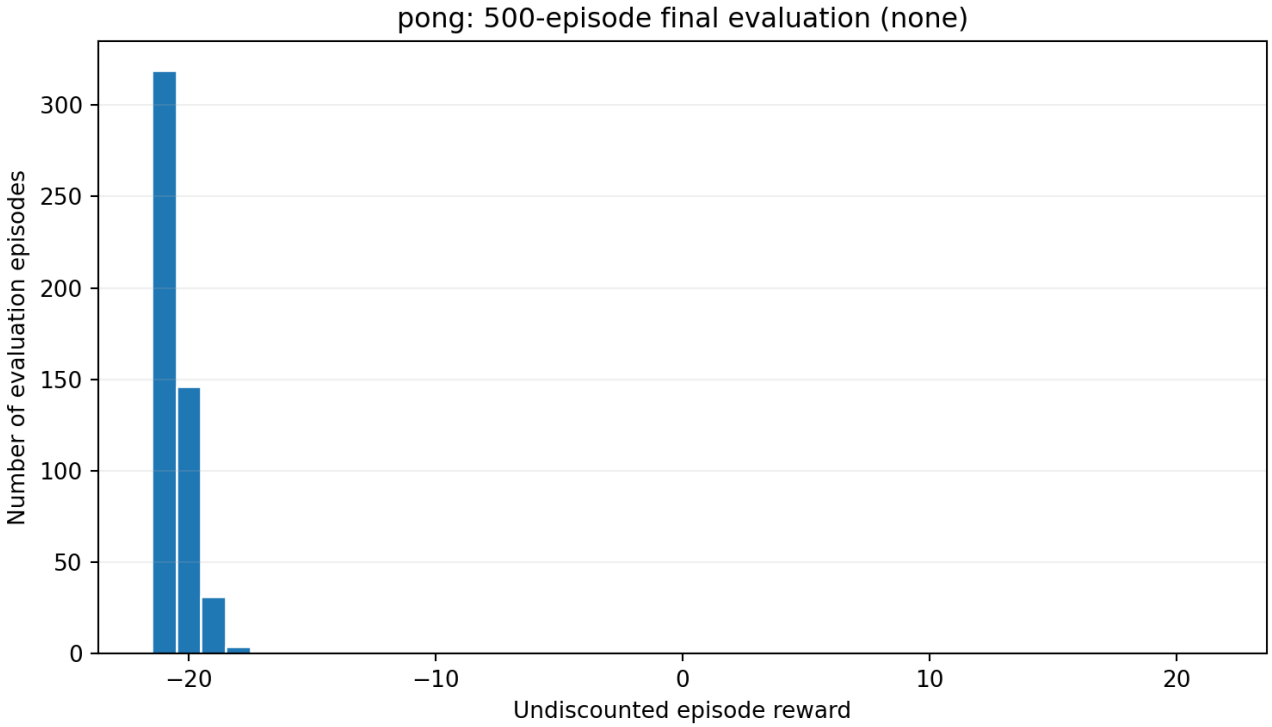
Pong: without baseline

Experimental setting	Observed result
Environment	ALE/Pong-v5

Experimental setting	Observed result
Training episodes	200
Discount gamma	0.99
Adam learning rate	0.0003
First 100 training episodes, mean reward	-20.340
Last 100 training episodes, mean reward	-20.580
500-episode evaluation mean	-20.560
500-episode evaluation sample std	0.647
Evaluation action method	stochastic sampling from two-action policy
95% normal CI for evaluation mean	-20.617 to -20.503
Artificially capped training / evaluation games	0 / 0



pong: reward per training episode and 100-episode moving average.



pong: empirical score distribution of 500 final policy rollouts.

4. Policy gradient: Part 2 (20 points)

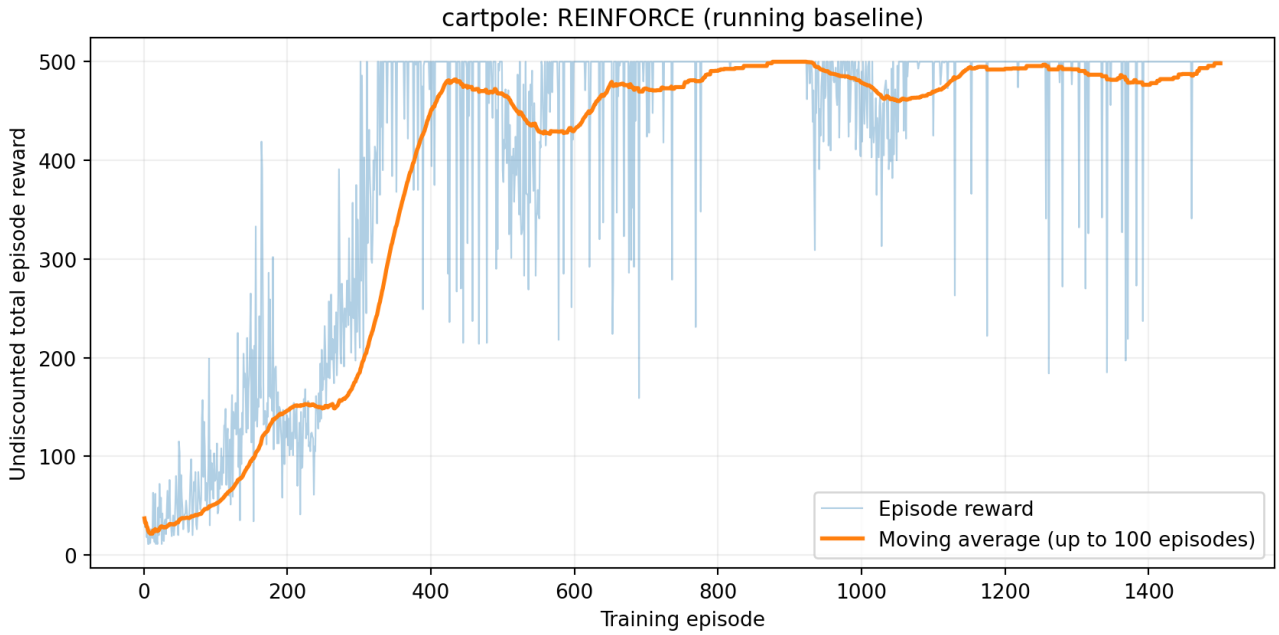
Chosen constant running-average baseline

The baseline for each policy update is a scalar exponential moving average of the mean discounted reward-to-go from PREVIOUS minibatches. Set b_1 to the first batch mean, then update b_(k+1) = 0.9*b_k + 0.1*mean(G_t in batch k). During batch k, each trajectory uses A_t = G_t - b_k in place of G_t. Because b_k is computed from prior episodes and does not depend on the current sampled action, subtracting the baseline does not change the expected score-function gradient. It may reduce its variance; it does not guarantee higher reward in every finite training run.

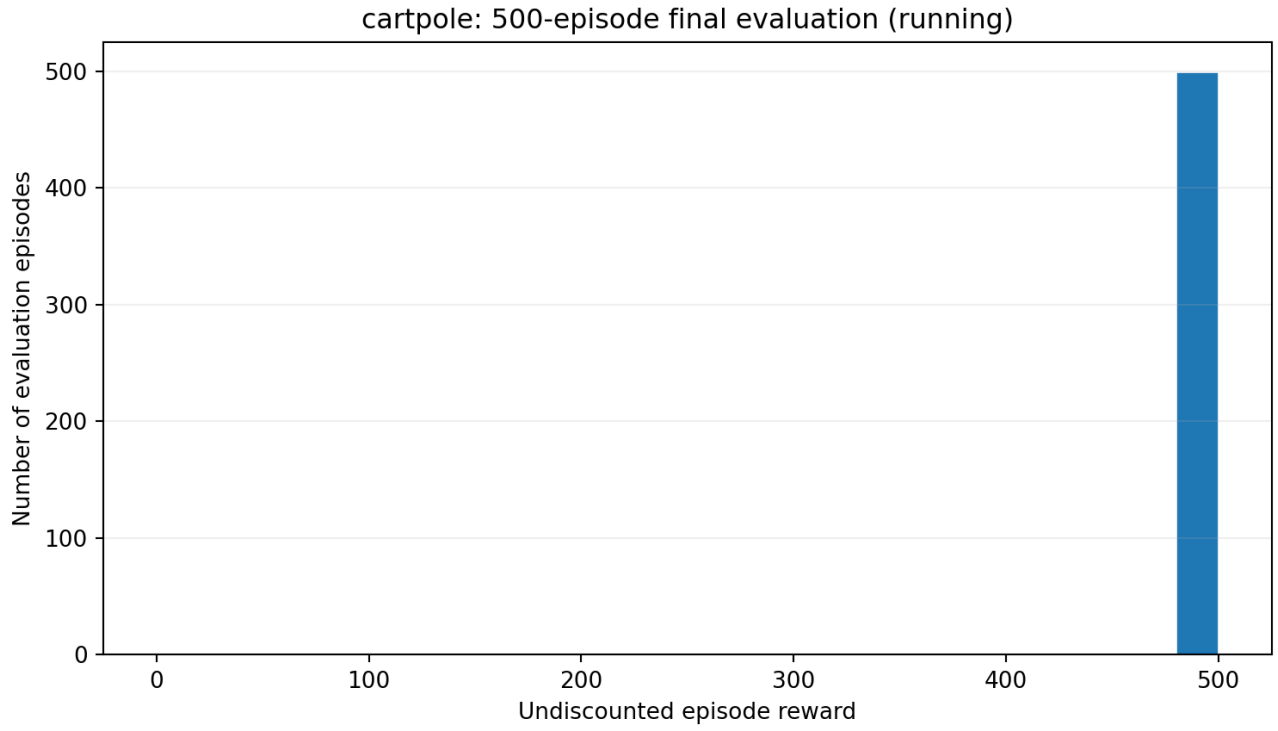
```
L_baseline(theta) = -(1/N) * sum_t log pi_theta(a_t|s_t) * (G_t - b)
```

Cartpole: with running baseline

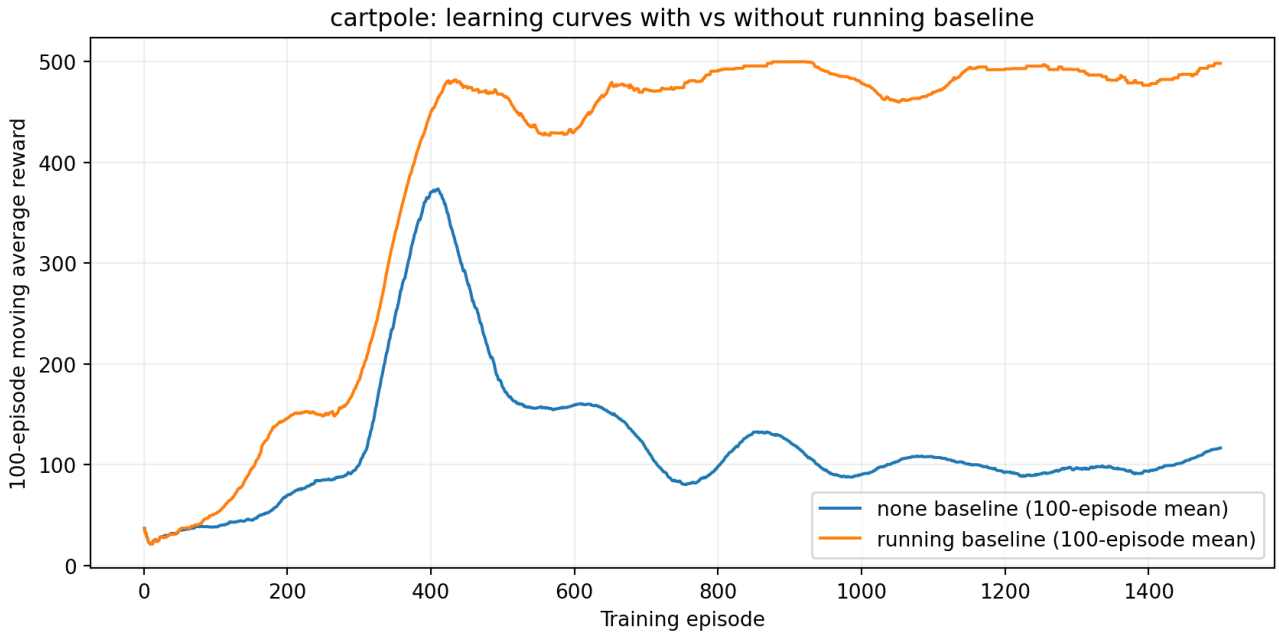
Metric	Observed value
First 100 training episodes, mean	51.540
Last 100 training episodes, mean	498.410
500-episode mean	500.000
500-episode sample standard deviation	0.000
Learned running baseline, final	19.227
Evaluation mean minus no-baseline mean	+373.376



cartpole: running-baseline training curve.



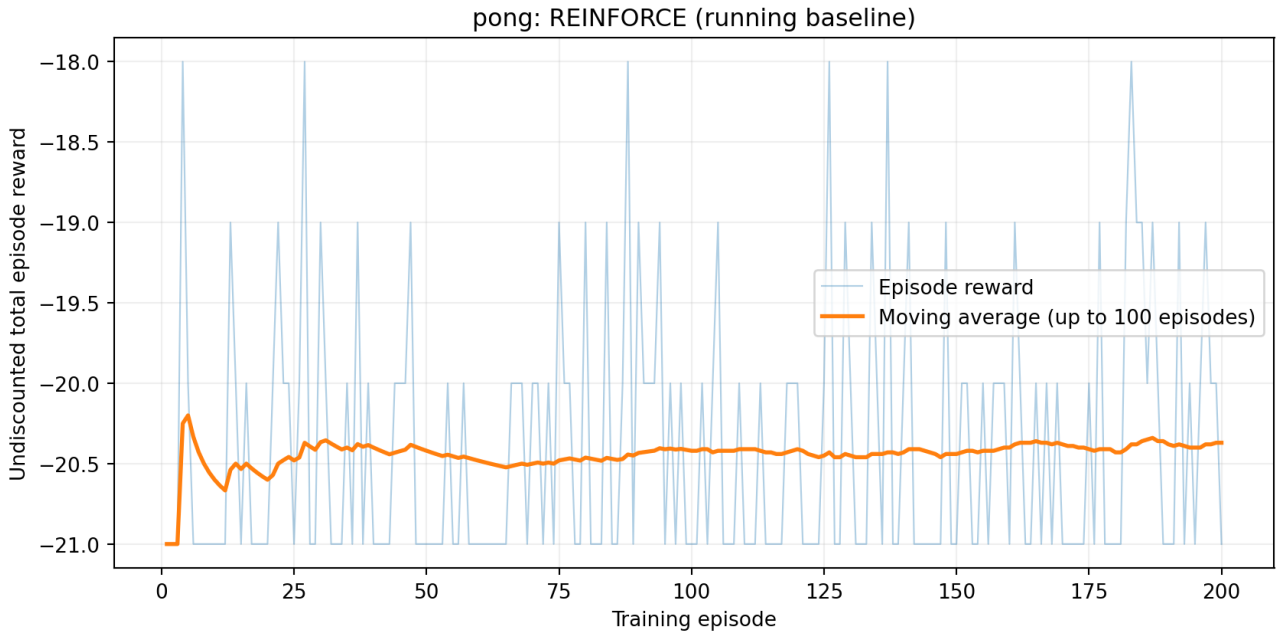
cartpole: 500-episode evaluation with baseline during training.



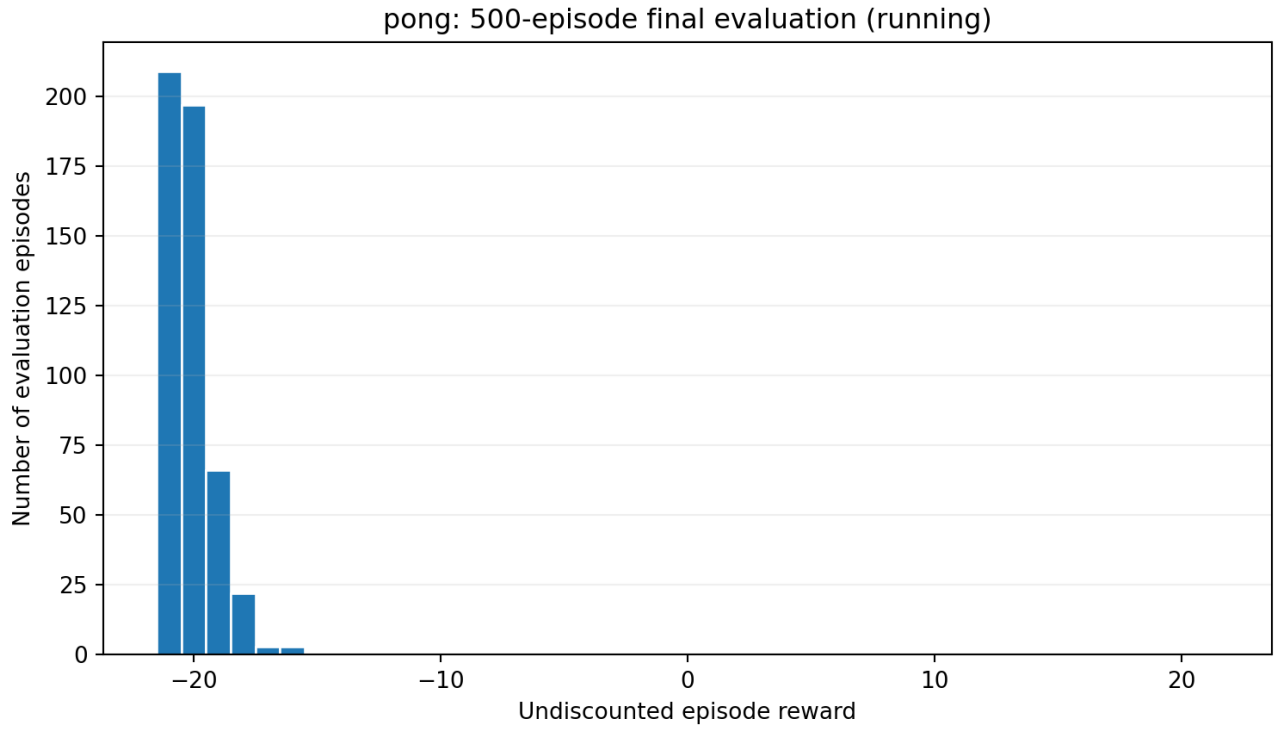
cartpole: 100-episode moving averages for both training runs.

Pong: with running baseline

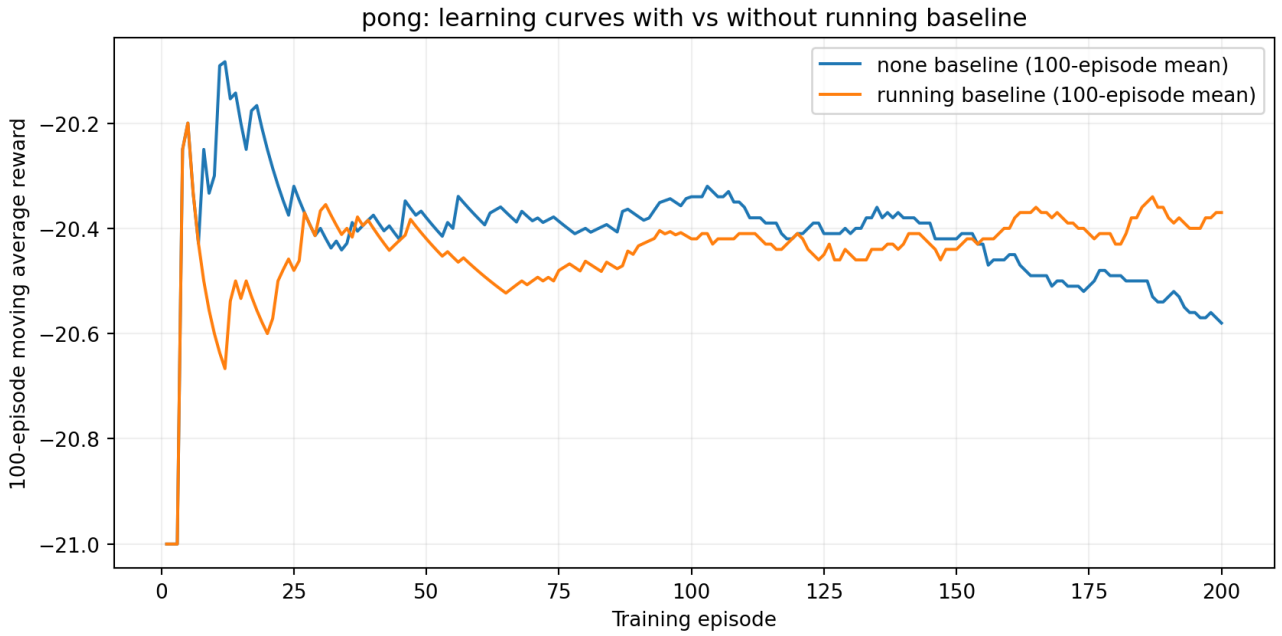
Metric	Observed value
First 100 training episodes, mean	-20.420
Last 100 training episodes, mean	-20.370
500-episode mean	-20.156
500-episode sample standard deviation	0.924
Learned running baseline, final	-1.725
Evaluation mean minus no-baseline mean	+0.404



pong: running-baseline training curve.



pong: 500-episode evaluation with baseline during training.



pong: 100-episode moving averages for both training runs.

5. Supplementary experiment: visually guided Pong

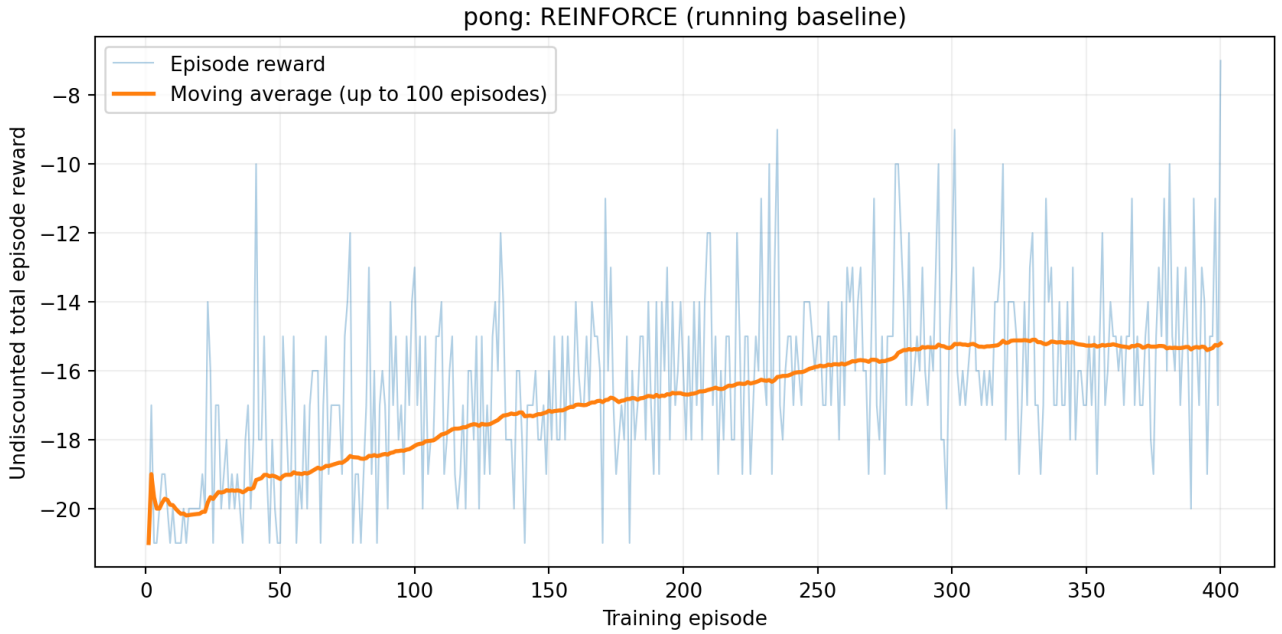
This supplementary model addresses the observed failure of reward-only Pong policies to learn useful ball tracking in the tested training budget. It shares the geometry-aware image policy but was trained using point/rally-level reward-to-go, updates after groups of five rallies, and an auxiliary direction-prediction loss from visible ball-paddle position. It is NOT an unchanged basic REINFORCE run and must NOT be credited as training solely from episode reward.

```
L_total = L_policy_gradient + 0.5 * L_visual_direction - 0.001 * entropy
```

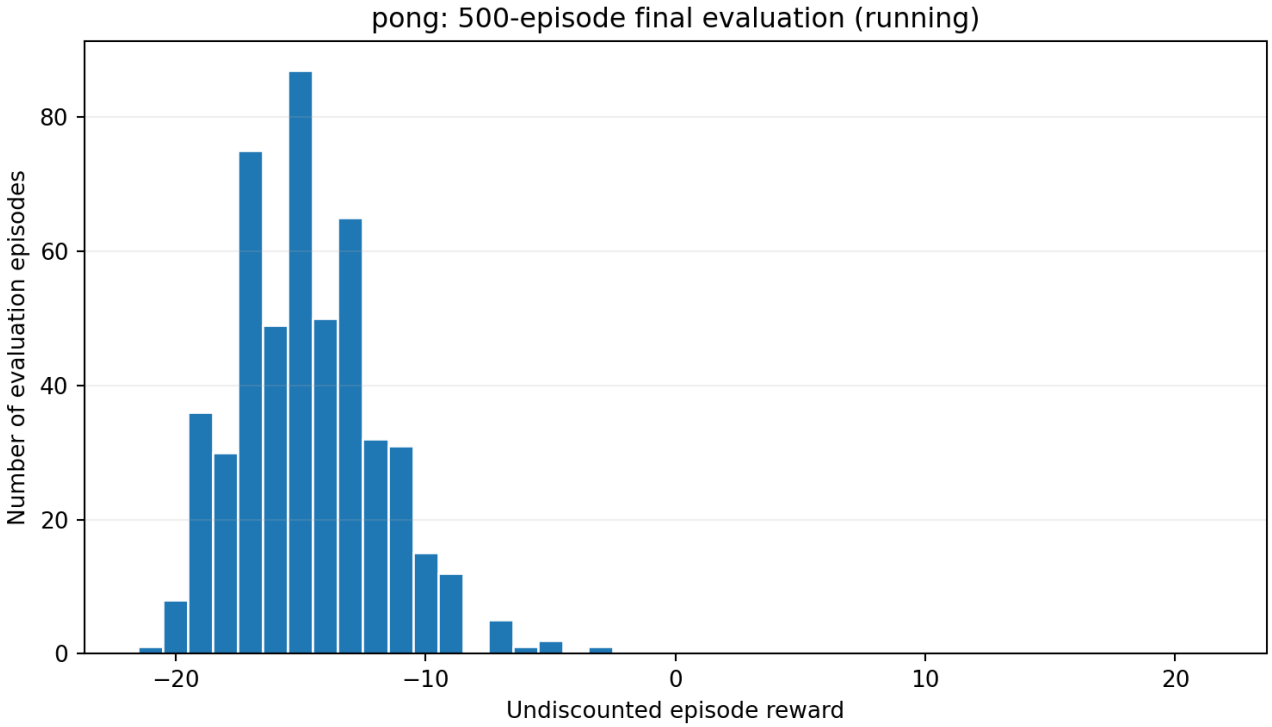
The direction targets come from image geometry; action 2 was calibrated to move the paddle UP. The CNN/geometry network uses pixels only; a fixed initial FIRE action starts play but is not selected by the network. All other learned actions are restricted to actions 2 and 3.

Measured setting	Guided Pong value
Training episodes	400
Reward discount	0.99
Visual guide coefficient	0.5
Entropy coefficient	0.001
Training mode	rally / point
Training optimizer updates	2217
First 100 training mean	-18.180
Last 100 training mean	-15.210

Measured setting	Guided Pong value
Evaluation episodes	500
Evaluation mean	-14.702
Evaluation sample std	2.843
95% normal CI for mean	-14.951 to -14.453
Observed minimum / maximum	-21 / -3
Mean policy entropy	0.146



Supplementary Pong: reward at each training episode and its 100-episode moving average.



Supplementary Pong: histogram over 500 complete stochastic-policy evaluation games.

Measured evaluation difference (guided minus reward-only baseline): +5.454 points. This is a comparison between different training objectives AND update schedules, so it is not a controlled estimate of the effect of visual guidance alone. Both methods were evaluated on 500 games; a follow-up ablation is needed to isolate each contributing change.

Despite improvement, the mean score remains negative and the agent has not demonstrated consistently winning games. This result illustrates the potential value of inductive visual structure and a denser learning signal under sparse-reward conditions.

6. Reinforcement-learning validation and edge cases

These checks complement (rather than replace) full experimental results. The deterministic checks are executable with pytest. The random-policy controls and full 500-episode evaluations require the actual Gymnasium environments.

• **Analytical return check:** for rewards [1,2,3] and gamma=0.5, reward-to-go must equal [2.75,3.5,3]. For gamma=0, it must equal immediate rewards. This tests the learning signal rather than only shapes.

• **Zero-reward / zero-advantage check:** multiplying selected log probabilities by zero should produce zero gradient.

• **Pong preprocessing check:** a synthetic 210x160x3 RGB frame produces binary 80x80 float input; background colors 109 and 144 are removed, while paddle/ball pixels survive.

• **Action constraints:** both policy heads emit exactly two logits; Pong policy actions map exclusively to 2 and 3; initial FIRE is environment initialization, not a policy decision.

• **Terminal/truncation boundaries:** each episode has its own discounted return calculation; rewards never propagate between different episodes. Training and evaluation count separately.

• **Finite learning behavior:** compare the means of the first 100 and last 100 training episodes, and compare trained results against randomized untrained-policy controls.

• **First ten steps:** stored in each metrics.json with state or image-difference statistics, action probabilities, selected environment action, reward and termination flag.

• **Data collection invariants:** 500 full evaluation games are required. The report lists any artificially capped games; a nonzero cap count invalidates a claim of 500 complete games.

cartpole random/untrained control: 20 episodes, sample mean 20.300, sample standard deviation 12.790.

cartpole trained comparison: without baseline mean 126.624 (std 5.694); with baseline mean 500.000 (std 0.000). These are outcomes under the same fixed evaluation seeds, not proof of improvement independent of training randomness.

pong random/untrained control: 5 episodes, sample mean -19.800, sample standard deviation 0.837.

pong trained comparison: without baseline mean -20.560 (std 0.647); with baseline mean -20.156 (std 0.924). These are outcomes under the same fixed evaluation seeds, not proof of improvement independent of training randomness.

7. What I would test with three more months

• Repeat every learning experiment over at least 10-20 independent training seeds. Report mean, standard deviation and confidence interval of final performance across seeds, not only environment stochasticity within a single trained policy.

• Perform a controlled grid search over learning rates, number of training episodes, batch sizes, network capacity and policy entropy. Report sample-efficiency curves (reward vs environment steps), wall-clock compute, and sensitivity to random initializations.

• Compare scalar baseline with a time-dependent baseline and a learned state-value critic while holding policy architecture and data budget fixed. Measure variance of per-update gradient estimates directly.

• For Pong, compare raw preprocessed frame versus consecutive-frame difference and 2/4 frame stacking; document firing/startup behavior, sticky-action stochasticity and full-game termination.

• For the shuttle MDP, challenge the arrival distribution, capacity constraint, terminal costs and holding-cost convention. Validate against direct policy enumeration on a very small state space and against very long Monte Carlo rollouts.

• For Pong, independently ablate the geometry-aware network, guided direction loss, point-level return truncation, rally-level update frequency, and entropy term. Keep training steps and evaluation seeds fixed to identify which changes cause the gain.

• Measure policy robustness after changing dynamics at test time (arrivals, Pong opponent behavior) and monitor overfitting or catastrophic learning changes with checkpoint evaluations.

8. Reproducibility and limitations

The experiments with no baseline and with running baseline use identical Pong reward-only configurations and the required two-action policy. In contrast, guided Pong is a supplementary hybrid objective, not a compliant replacement for either required condition. Results for a single trained seed and 500 environment episodes describe conditional performance, not the variation across independently trained policies.

Source code contains the complete algorithm, deterministic tests, CSV records of per-episode rewards, first-ten-step action traces, trained-policy checkpoints, and automatically generated plots. The runs use the documented fixed seed 2026 by default. Pong training can be computationally expensive and the assignment does not promise it will converge to a winning policy at the selected compute budget. All results should be evaluated on the actual required game versions, not on substitute simulations.

Environment references: Gymnasium CartPole-v1 and Arcade Learning Environment Pong-v5 (https://gymnasium.farama.org/; https://ale.farama.org/). Assignment assumptions are documented in Section 2. The code is implemented using standard pip-installable packages.